# Gateway outbound destination control

Architecture, interfaces, and delivery requirements

Miguel Branco · 28 October 2026

**Destination authorization only**
Native credentials and end-to-end TLS remain unchanged. This RFC adds no runtime implementation.

## Contents

1 Control Gateway outbound connections through the selected proxy ..... 2
1.1 Problem and decision ..... 2
1.2 Operator behavior ..... 2
1.3 Owners and contract changes ..... 2
1.4 Network and lifecycle requirements ..... 2
1.5 Placement and delivery boundary ..... 3
2 Gateway outbound architecture and interface report ..... 4
2.1 Scope and baseline ..... 4
2.2 One request ..... 4
2.3 What changes from main ..... 5
2.4 Shared infrastructure and workload counts ..... 5
2.5 Current interfaces and callers ..... 5
2.6 OpenShell compatibility ..... 6
2.7 Authorization and lifecycle ..... 7
2.8 Verification and implementation status ..... 7
3 Gateway outbound source catalogue ..... 8
3.1 Selected OpenShell release ..... 8

# 1 Control Gateway outbound connections through the selected proxy

## 1.1 Problem and decision

A dedicated OpenClaw Gateway runs trusted plugins, channel clients, and other integrations. Its Kubernetes network policy permits specific internal peers and a configured channel proxy. The bundled proxy admits Slack destinations, not arbitrary administrator-approved services. For example, a Gateway cannot use ClawHub merely because its administrator enabled the relevant software.

Add Gateway destination authorization to the selected Credential Proxy capability. Reuse the selected implementation's network proxy and, where needed, Sandbox networking. Do not add another OCE forwarding implementation or proxy Deployment. OpenClaw Control Plane (OCC) owns the permission; the selected Driver owns enforcement; Compute owns workload routing and lifecycle.

This is transport control, not credential mediation. Real integration credentials remain in the Gateway, and dedicated Harness authentication remains unchanged. TLS stays end to end. The proxy does not replace credentials, inspect HTTP content, authorize provider actions, or filter responses.

## 1.2 Operator behavior

The proposed OCE Agent setting `gatewayOutbound` selects access for a dedicated Gateway, separately from native OpenClaw Configuration:

| Selection | Gateway access |
|---|---|
| `channelsOnly` | Enabled channels' declared destinations only. |
| `restricted` | Channel destinations plus exact administrator-approved DNS hosts on TCP/443. |
| `publicHttps` | Public TCP/443, subject to the Installation ceiling and forbidden addresses. |

Public mode is disabled unless the Installation permits it. TCP/443 access is not proof that the client uses HTTPS. The setting is not a plugin identity or permission to use another integration's credentials.

Changing this setting requires `update` and `administer` on the exact Agent. Deployment retains its existing authorization checks. OCC records attributable mutations and authorization denials and freezes the selection, channel requirements, and Installation ceiling in the Agent revision. Configuration changes cannot widen the selected access.

Ordinary embedded execution keeps its existing network behavior, credentials, storage, and placement. This proposal adds no embedded policy selection or automatic sandbox. A dedicated Agent without the new capability selected keeps its existing supported routing; it does not gain general outbound access.

## 1.3 Owners and contract changes

These additions are proposed contracts, not existing APIs:

- **Channel Driver:** contribute nonsecret destination requirements for its enabled configuration. Keep Gateway runtime requirements separate from control-plane credential validation and directory lookup.
- **Credential Proxy Driver:** advertise destination-only Gateway support; admit and install a revision-scoped destination policy, report its effective installation, and withdraw its connections. Keep this capability separate from credential-bound `ProxyPlan` routes and request-exchange callbacks.
- **Compute Driver:** advertise compatible routing, connect only the intended Gateway to its policy, and enforce that external traffic cannot bypass the selected proxy. Preserve exact Harness and platform transport peers.

OCC rejects an explicit selection when either selected Driver lacks support. It must not create a dummy Secret, credential source, or protected credential grant to represent a destination permission. Policies for two Agents must not share authority merely because they use the same proxy service.

The current Credential Proxy contract is credential/source-bound, and the current Sandbox contract provisions Harnesses. Neither already provides this Gateway-facing transport. The bundled OpenShell integration therefore needs a supported transport capability before runtime delivery. Its existing sandbox destination rules are a reusable enforcement primitive, not evidence that an ordinary Gateway can join that path.

## 1.4 Network and lifecycle requirements

Destination rules authorize host and port, not methods, paths, bodies, or individual plugins. Channel Drivers may declare necessary DNS suffixes; manual restricted grants name exact DNS hosts. The Installation excludes private, loopback, link-local, metadata, reserved, and actual infrastructure addresses. The selected proxy must validate resolved addresses and connect only to an approved result. A permitted hostname cannot override an address exclusion.

The initial delivery requires IPv4 transport and TCP/443. Unsupported addresses, protocols, and clients fail explicitly; there is no direct-network fallback. Native certificate and hostname verification remain enabled. An external redirect that requires a new connection receives another destination check.

Compute installs and observes the exact Gateway policy before activating its route. A missing proxy, ambiguous workload identity, or unconfirmed policy keeps the new revision unready. Changing desired state takes effect through deployment. Disabling a channel removes only that channel's contribution; independently approved access remains.

Withdrawal must deny new connections and close the affected established connections before reporting completion. Replacement and cleanup must identify the exact revision, fence its access, and preserve the successor. A shared proxy is acceptable only if it can identify, isolate, and withdraw each Gateway's connections. Proxy failure never enables a broader route.

## 1.5 Placement and delivery boundary

This decision does not require moving the Gateway into a sandbox, adding a sandbox per Gateway, changing runtime images, or changing dedicated Harness sandboxing. Those are not consequences of reusing a Driver registration. An implementation must establish a supported Gateway-facing route using the existing primitives. If that requires a separate placement decision or supplier change, bring that decision for review instead of adding it to this delivery.

This RFC and its architecture report are the complete change. They add no runtime implementation or supported configuration. Implementation requires the Gateway-facing transport first, then the normal Agent API/worker workflow and real allowed/denied destination, direct-bypass, cross-Agent, withdrawal, and replacement proof. Ordinary embedded execution and credential delivery must remain unchanged. See source evidence.

# 2 Gateway outbound architecture and interface report

## 2.1 Scope and baseline

This report compares the proposed destination-control design with upstream `main` at `5dae426ec5c591b43609a4b50c154016e2c5483c`, inspected on 28 October 2026. The change contains the RFC and reading materials only. Source inspection establishes current contracts and gaps; it is not a runtime test or a deployment qualification.

The task is to let an administrator control where trusted Gateway software can connect, using existing proxy primitives. It does not change which program holds a credential, how a provider authenticates a request, or which provider actions the credential permits.

## 2.2 One request

An administrator approves `clawhub.ai` for a dedicated Gateway and deploys its Agent. The Gateway's normal ClawHub client requests the service through the selected network proxy. The proxy identifies the Gateway, checks its admitted destination policy and resolved addresses, and forwards the permitted TCP/443 connection. A request to an unapproved host fails. Direct connections cannot bypass the proxy's policy.

The client performs the TLS handshake with ClawHub and verifies its certificate. The forwarding path does not terminate that TLS connection. If an integration uses an authorization header, its Gateway client still supplies the real credential as it does today. This design neither reads nor rewrites that header.

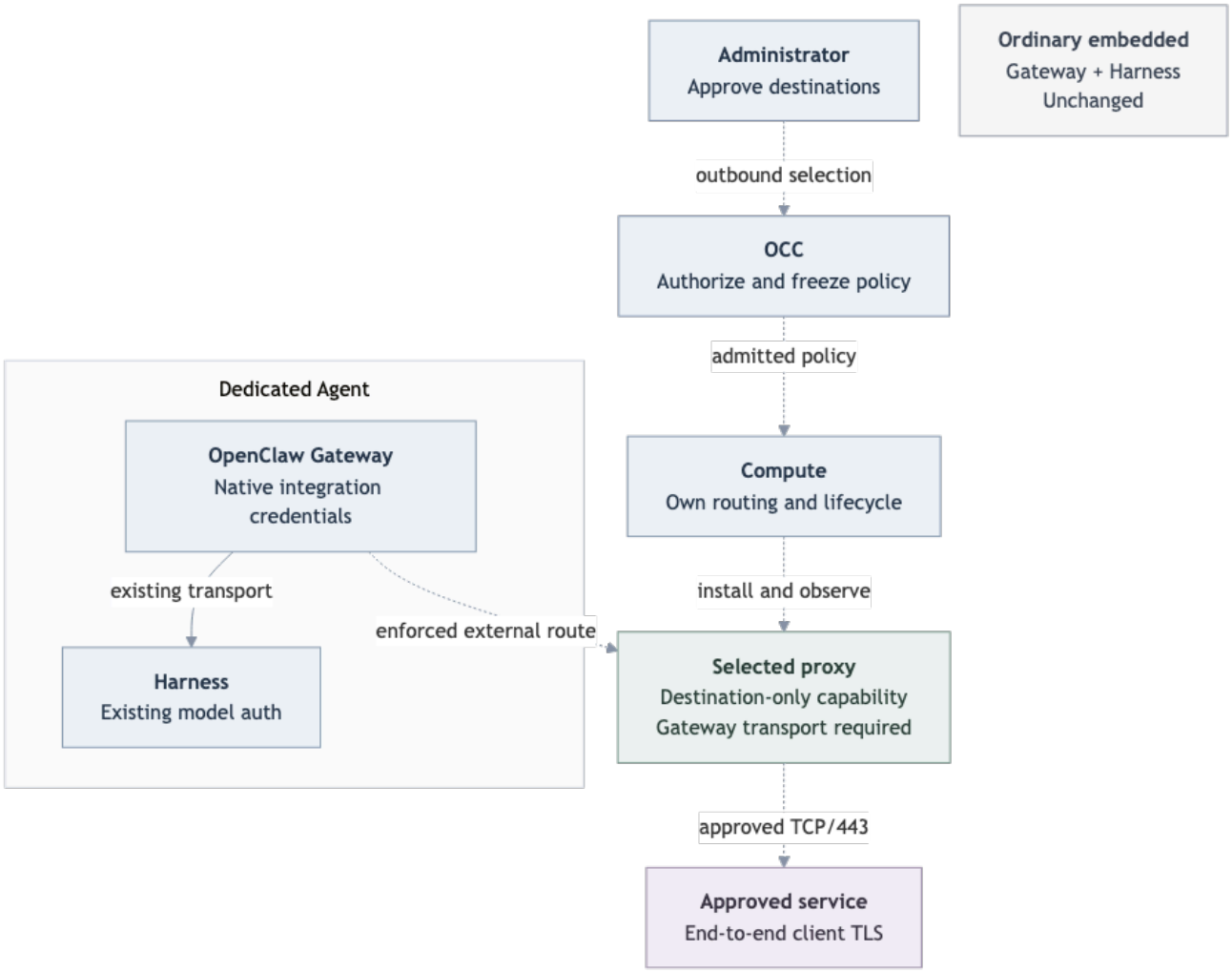



Figure 1: Proposed destination-control path. Dashed edges require implementation; solid edges retain existing ownership.

The policy applies to the whole Gateway. Trusted plugins in that process can still read its credentials and use any destination granted to the Gateway. Host approval does not provide plugin isolation, HTTP operation restrictions, or protection against misuse of an approved API.

## 2.3 What changes from main

| Area | Current main | Proposed change |
|---|---|---|
| Administrator intent | No general Agent Gateway destination selection. | Add a Gateway destination selection and Installation ceiling. |
| Channel destinations | Compute contains fixed channel requirements. | Selected Channel Drivers contribute their requirements. |
| Gateway enforcement | A configured channel proxy; bundled CONNECT forwarding admits Slack hosts. | The selected proxy enforces the Gateway's admitted destination policy. |
| Proxy contracts | Credential/source-bound protected execution. | Add an optional destination-only Gateway capability. |
| Gateway credentials | Native integration credentials enter the Gateway workload. | No change. |
| Dedicated model authentication | Existing Harness authentication path; the dedicated Gateway receives no Harness inference credential. | No change. |
| TLS and application requests | Native client authentication and TLS verification. | No credential substitution, TLS interception, or response rewriting. |
| Ordinary embedded execution | Combined Gateway/Harness with its supported network and credential configuration. | No change; no new policy selection or sandbox. |
| Runtime packaging and infrastructure | Selected Compute/Sandbox/proxy implementations. | No new forwarding implementation, proxy Deployment, image version, or supplier patch in this change. |

The setting names in the RFC are proposed OCE API fields. They are not supported on current main. This document does not promote them into the current feature reference or claim that selecting an existing proxy enables them today.

## 2.4 Shared infrastructure and workload counts

A shared OpenShell service, a Driver registration, an enforcement process, and a workload sandbox are different things. Main selects one bundled OpenShell Backend; that service can manage multiple independent Harness sandboxes. Each sandbox has a distinct Sandbox identity and policy. This does not imply a separate Kubernetes ServiceAccount for each Agent.

For two dedicated Codex Agents with one running revision each:

- **Without the optional OpenShell Harness placement:** main does not require OpenShell sandboxes for those Agents.
- **With OpenShell Harness placement:** there are two Harness sandboxes, one for each Codex execution. Each OpenClaw Gateway remains outside that sandbox.
- **Under this proposal:** neither Harness placement changes, and no additional Gateway sandbox is mandated. The enforcing adapter's supported transport will determine its process and connection ownership. No fixed extra-Pod count is established by this RFC.

For ordinary embedded OpenClaw, the Gateway and Harness continue to share their existing workload. There is no new OpenShell sandbox and no automatic conversion to dedicated execution. Existing network restrictions and supported channel topology still apply; "unchanged" does not mean unrestricted access to cluster or metadata addresses.

## 2.5 Current interfaces and callers

The following declarations exist on the pinned main. The source catalogue links their complete definitions.

### 2.5.1 Channel Driver

```
validateCredentials?(
  values: Readonly<Record<string, unknown>>,
  withSecret: ChannelCredentialReader,
): Promise<void>;

lookupDirectory(
  input: ChannelDirectoryLookupInput,
  signal?: AbortSignal,
): Promise<ChannelDirectoryResult>;
```

These methods serve control-plane setup. Neither declares Gateway runtime destinations. The proposal adds a nonsecret destination contribution for the enabled channel configuration. It must distinguish those runtime destinations from setup traffic; an Agent grant cannot widen control-plane network access.

### 2.5.2 Credential Proxy Driver

```
configureRevision(context: ProxyRevisionContext): Promise<ProxyWiring>;

wiringStatus(
  context: ProxyRevisionContext,
  prepared: ProxyWiring,
): Promise<EgressInstallation | undefined>;

verifyInstallation(
  context: ProxyRevisionContext,
  installation: EgressInstallation,
): Promise<boolean>;

removeRevision(context: ProxyRevisionContext): Promise<boolean>;
```

These methods configure and verify an execution's credential-bound plan. `ProxyPlan.routes` contains source/binding/grant records. The bundled OpenShell adapter limits an ordinary protected plan to one binding and one literal host, then enables credential and execution authority middleware. It is not an arbitrary Gateway destination-policy API.

The proposed optional capability needs four behaviors: admit/install a destination policy, observe its effective installation, recheck that the correct Gateway still uses it, and withdraw its connections. Its input contains the Gateway's revision identity and nonsecret destination rules, not a credential source. It must not reuse credential-bound plans by inventing empty sources or tokens. Concrete method signatures belong to the supported transport design; the required behaviors do not depend on a particular supplier protocol.

The existing exchange callbacks `resolve`, `authorizeSend`, `beginResponse`, `renewResponse`, and `protectResponse` are **not requirements of this delivery**. Destination authorization needs no Secret read, token preparation, credential installation, provider adapter, or response-custody protocol.

### 2.5.3 Compute and Sandbox

Compute owns the Gateway workload, Secrets, baseline network isolation, routing, and activation. The selected proxy owns its Gateway destination rules and connection lifecycle. Sandbox retains its existing Harness and network-policy responsibilities. OCC calls these capabilities through their platform contracts; it does not add OpenShell branches to core admission.

The current Sandbox method `provisionHarness(context)` creates a dedicated Harness. `harnessEndpoint(context)` and `harnessStatus(context)` identify and check that Harness. They do not provision or identify a Gateway. Reusing their implementation cannot justify presenting a Gateway as a Harness.

## 2.6 OpenShell compatibility

OpenShell's network-policy rules already express executable, host, and port permissions. OCE renders ordinary Sandbox additions with `tls: NETWORK_TLS_MODE_SKIP`. This supplies a destination-control primitive without requiring credential rewriting or HTTPS inspection.

That primitive operates inside an OpenShell-managed sandbox. It does not expose a supported remote forwarding endpoint that an arbitrary existing Gateway Pod can join. Current OCE's OpenShell adapter provisions dedicated Harnesses; it does not wrap an ordinary Gateway merely because the same Backend is selected.

**The missing Gateway-facing transport is an implementation dependency.** The destination-only capability must not be advertised until a supported adapter can authenticate the Gateway, bind its policy, and prevent direct bypass while preserving the intended workload and credential boundaries. Stock OpenShell's existing functionality alone does not establish this path.

Moving the Gateway workload into an OpenShell sandbox could supply a network boundary, but it would also require placement, Secret projection, storage, routing, and lifecycle work. This RFC does not select that expansion. It also does not patch OpenShell, package its internal host-proxy library as a new service, or silently replace the requested destination-only design with protected credential execution. If no supported adapter meets the contract, runtime delivery remains unavailable.

## 2.7 Authorization and lifecycle

1. An authorized administrator changes the exact Agent's outbound selection. OCC checks `update` and `administer` and records the attributable decision.
2. Deployment checks current authority and the selected Compute/proxy capabilities, then freezes the policy and channel contributions in the revision. Unsupported explicit selections fail; they do not choose another execution mode.
3. Compute prepares the existing Gateway and its required private peers. The selected proxy installs the exact revision's destination rules. Compute observes effective enforcement before activating the route.
4. The proxy checks each new external connection against the correct Gateway policy and address exclusions. An allowed destination carries the native client's encrypted traffic; it is not a provider-operation authorization.
5. Desired-state changes take effect through deployment. Removing a channel removes only that contribution. An independent administrator grant can remain.
6. Stop, withdrawal, and replacement close the affected connections and remove only the intended revision's access. Unconfirmed cleanup remains pending. Proxy failure never grants a direct route.

No new token-renewal loop, credential store, migration, or persistent provider session protocol is part of this delivery. Implementation must use existing revision ownership and worker reconciliation rather than inventing a second authorization or lifecycle system.

## 2.8 Verification and implementation status

The PR contains no runtime implementation. Required behavioral evidence is therefore **pending**, not passed or skipped. Documentation builds, source-link checks, PDF inspection, and independent review establish only the document's consistency and scope.

A later implementation must extend the real Agent API/worker workflow with the selected supported proxy and enforce these observable outcomes:

| Required case | Observable outcome |
|---|---|
| Approved and unapproved host | The real Gateway caller succeeds only for the approved destination. |
| Channel contribution | Enabling and disabling a channel changes its contribution at deployment without deleting independent grants. |
| Direct bypass and forbidden address | Direct sockets and DNS answers naming excluded infrastructure cannot bypass policy. |
| Two Agents | One Gateway cannot use the other's rules or connection identity. |
| Proxy failure and unknown installation | New connections or activation fail; no broader route opens. |
| Withdrawal and replacement | Affected connections close; stale cleanup cannot remove the successor's access. |
| Native credentials and TLS | A real integration still uses its native credential from the Gateway; certificate verification stays enabled and the proxy does not rewrite it. |
| Ordinary embedded execution | Existing credential delivery, network behavior, and placement remain unchanged. |

Use the repository's integration requirements. A mocked policy decision, direct helper call, or successful startup alone cannot prove the normal Gateway path. No model, credential, provider, or production qualification is claimed here.

# 3 Gateway outbound source catalogue

All implementation links below point to upstream main at `5dae426ec5c591b43609a4b50c154016e2c5483c`. They establish current code and contracts, not a deployed Gateway destination-control feature. Read the RFC and architecture report for the proposal.

| Source | Evidence |
|---|---|
| Architecture and implementation status | Current ownership, placement, and remaining platform work. |
| Harness authentication | Embedded model Secrets enter the combined workload. Dedicated Harness authentication uses its existing direct or proxy path; the Gateway receives no Harness inference credential. |
| Kubernetes network profiles | Ordinary workload and provider-fenced Harness networking are distinct. |
| Bundled Slack CONNECT proxy | Fixed Slack host/suffix checks on port 443 and opaque stream forwarding. |
| Kubernetes Compute | Gateway workload, credentials, channel proxy routing, isolation, and lifecycle. |
| Channel and Sandbox contracts | `ChannelDriver` has setup/directory methods; `SandboxDriver` has Harness provisioning and network-policy operations. |
| Credential Proxy contract | Current `ProxyPlan`, `CredentialProxyDriver`, configuration, observation, verification, and removal declarations. |
| Protected execution caller | Current orchestration requires Harness-specific provisioning and enrollment. |
| OpenShell proxy adapter | Source-bound protected policy, one-host/one-binding restrictions, and credential/execution middleware. |
| OpenShell capability registration | Existing selected Backend pairs Sandbox and proxy capabilities. |
| OpenShell destination rules | Ordinary Sandbox network additions use host/port/binary rules with TLS inspection skipped. |
| OpenShell Sandbox adapter | Current workload admission, Harness provisioning, and projection limits. |
| OpenShell ownership and limits | Per-Harness sandbox ownership and current qualification boundary. |
| Backend configuration | One bundled OpenShell Backend selection, not one shared execution sandbox. |

The proposal adds no implementation declarations to these files. In particular, the current contracts do not contain the proposed Gateway destination selection, Channel destination contribution, or Gateway-facing destination-only proxy port.

## 3.1 Selected OpenShell release

The stock `v0.1.5` source archive matches the checksum selected by main:
`b627a526427197854f8ee1d783996e666e0bfd6b7a785b70df7709304d9b6f62`.

- TLS-skip forwarding uses a raw tunnel without TLS termination or credential injection.
- Destination policy matches hosts and ports within the sandbox's policy.
- Upstream address validation connects using the validated addresses.
- Host-side proxy entry point is a Rust integration API with loopback binding and no middleware support, not an existing remote Gateway-facing service.

These sources establish the reusable forwarding primitive and its limits. They do not establish the missing OCE Gateway transport.